1. Análise de resultados de alunos

Cada aluno é representado por um dicionário. As notas estão numa lista de tuplas, em que cada tupla contém o nome da disciplina e a respetiva nota.

```
alunos = [
    {
        "numero": 101,
        "nome": "Ana",
        "notas": [("Matemática", 16), ("Programação", 18), ("Inglês", 14)]
    },
    {
        "numero": 102,
        "nome": "Bruno",
        "notas": [("Matemática", 9), ("Programação", 15), ("Inglês", 13)]
    },
    {
        "numero": 103,
        "nome": "Carla",
        "notas": [("Matemática", 17), ("Programação", 16), ("Inglês", 18)]
    }
]
```

1. Cria um dicionário que associe o número de cada aluno à sua média, arredondada a duas casas decimais.

2. Produz uma lista de tuplas (nome, media), ordenada por média decrescente. Em caso de empate, ordena pelo nome.

3. Calcula a média de cada disciplina.

4. Identifica os alunos com pelo menos uma nota inferior a 10.

5. Identifica o melhor aluno de cada disciplina. Se houver empate, inclui todos os nomes.

A lista ordenada de médias deve ser:

```
[("Carla", 17.0), ("Ana", 16.0), ("Bruno", 12.33)]
```

2. Gestão de inventário com histórico de movimentos

O inventário associa cada código de produto a um dicionário com os seus dados. Os movimentos são tuplas no formato (codigo, tipo, quantidade).

```
inventario = {
    "P01": {"nome": "Teclado", "preco": 30.0, "stock": 10},
    "P02": {"nome": "Rato", "preco": 15.0, "stock": 8},
    "P03": {"nome": "Monitor", "preco": 180.0, "stock": 4}
}

movimentos = [
    ("P01", "saida", 3),
    ("P02", "entrada", 5),
    ("P03", "saida", 6),
    ("P04", "entrada", 2),
    ("P01", "entrada", 4)
]
```

1. Implementa uma função que processe os movimentos pela ordem apresentada.

2. Rejeita movimentos com produto inexistente, tipo inválido, quantidade não positiva ou saída superior ao stock disponível.

3. Guarda os movimentos rejeitados numa lista de dicionários com o movimento original e o motivo da rejeição.

4. Devolve um novo inventário, sem alterar o original nem os seus dicionários internos.

5. Calcula o valor total do inventário atualizado e lista os produtos com stock inferior a 5.

O stock final deve ser:

```
{"P01": 11, "P02": 13, "P03": 4}
```

Devem existir dois movimentos rejeitados.

3. Consolidação de transações e classificação de clientes

Cada transação é uma tupla com o formato (id_transacao, cliente, categoria, valor_centimos). Usam-se valores inteiros em cêntimos.

```
transacoes = [
    ("T01", "Ana", "Livros", 2500),
    ("T02", "Bruno", "Tecnologia", 8000),
    ("T03", "Ana", "Tecnologia", 5000),
    ("T02", "Bruno", "Tecnologia", 8000),
    ("T04", "Carla", "Livros", 3000),
    ("T05", "Bruno", "Livros", 2000),
    ("T06", "Ana", "Livros", 1500)
]
```

1. Remove transações duplicadas pelo identificador, preservando a primeira ocorrência e a ordem original.

2. Constrói um dicionário com o total gasto por cliente.

3. Constrói um dicionário aninhado no formato: {"Ana": {"Livros": 4000, "Tecnologia": 5000}}.

4. Produz uma lista de tuplas (cliente, total) ordenada por total decrescente e, nos empates, por nome.

5. Determina a categoria com maior faturação. Se houver empate, devolve todas as categorias empatadas.

Após remover os duplicados, os totais devem ser:

```
{"Ana": 9000, "Bruno": 10000, "Carla": 3000}
```

4. Deteção de conflitos em reservas de salas

Cada reserva é uma tupla (sala, inicio, fim, responsavel). Os horários são números inteiros que representam horas do mesmo dia.

```
reservas = [
    ("A", 9, 11, "Ana"),
    ("A", 10, 12, "Bruno"),
    ("A", 12, 14, "Carla"),
    ("B", 9, 10, "Daniel"),
    ("B", 10, 12, "Eva"),
    ("B", 11, 13, "Filipe")
]
```

1. Agrupa as reservas num dicionário cuja chave seja a sala e cujo valor seja a lista das respetivas reservas.

2. Ordena as reservas de cada sala pela hora de início.

3. Identifica todos os pares de reservas em conflito na mesma sala.

4. Considera os intervalos como [inicio, fim): uma reserva que termina às 10 não entra em conflito com outra que começa às 10.

5. Calcula o tempo total de ocupação de cada sala, contando apenas uma vez os períodos sobrepostos.

Existem dois pares em conflito: Ana–Bruno e Eva–Filipe.

Os tempos de ocupação, em horas, são:

```
{"A": 5, "B": 4}
```

5. Recomendação de itens com base em avaliações

As avaliações estão organizadas num dicionário que associa cada utilizador a uma lista de tuplas (item, nota). As notas variam entre 1 e 5.

```
avaliacoes = {
    "Ana": [("Livro A", 5), ("Livro B", 3)],
    "Bruno": [("Livro A", 4), ("Livro C", 5), ("Livro D", 2)],
    "Carla": [("Livro B", 4), ("Livro C", 4), ("Livro D", 5)],
    "Daniel": [("Livro A", 3), ("Livro C", 3), ("Livro E", 5)]
}
```

1. Inverte a estrutura para obter um dicionário que associe cada item à lista das suas avaliações.

2. Calcula, para cada item, a média e o número de avaliações.

3. Implementa a função recomendar(avaliacoes, utilizador, minimo_avaliacoes=2, limite=3).

4. A função deve excluir os itens já avaliados pelo utilizador e os que não atingem o número mínimo de avaliações.

5. Ordena os candidatos por média decrescente, depois por número de avaliações decrescente e, finalmente, pelo nome do item.

6. Devolve até limite tuplas (item, media, numero_avaliacoes). Para um utilizador inexistente, considera que ainda não avaliou nenhum item.

Para Ana, com os parâmetros predefinidos, o resultado deve ser:

```
[("Livro C", 4.0, 3), ("Livro D", 3.5, 2)]
```

6. Análise de classificações com opções de filtragem

Implementa a função:

```
def analisar_notas(*notas, **opcoes):
       ...
```

A função recebe um número variável de notas como argumentos posicionais. Os argumentos por palavra-chave permitem configurar a análise:

- minimo: menor nota válida; por omissão, 0.
- maximo: maior nota válida; por omissão, 20.
- aprovacao: nota mínima de aprovação; por omissão, 10.
- excluir_extremos: indica se deve ser removida uma ocorrência da menor e da maior nota válida; por omissão, False.

1. Separa as notas válidas das inválidas, preservando a ordem de entrada.

2. Se excluir_extremos=True e existirem pelo menos três notas válidas, remove uma ocorrência de cada extremo.

3. Calcula a média e as quantidades de aprovações e reprovações apenas sobre as notas que permanecem na análise.

4. Devolve três resultados: uma lista com as notas analisadas; um dicionário com a média e as quantidades de aprovações e reprovações; e uma lista com as notas inválidas.

Se não restarem notas para analisar, a média deve ser None e as contagens devem ser zero. Rejeita opções desconhecidas com ValueError.

```
notas, resumo, invalidas = analisar_notas(
       12, 18, 7, 25, 15,
       aprovacao=10,
       excluir_extremos=True
 )
```

Resultado esperado:

```
notas == [12, 15]

 resumo == {
       "media": 13.5,
       "aprovacoes": 2,
       "reprovacoes": 0
 }

 invalidas == [25]
```

7. Classificação de participantes com critérios variáveis

Implementa a função:

```
def classificar_participantes(*participantes, **pesos):
       ...
```

Cada argumento posicional é um dicionário com um nome e as pontuações obtidas em diferentes critérios. Cada argumento por palavra-chave identifica um critério e o respetivo peso.

```
ana = {"nome": "Ana", "teoria": 16, "pratica": 18}
 bruno = {"nome": "Bruno", "teoria": 18, "pratica": 14}
 carla = {"nome": "Carla", "teoria": 15}
```

1. Calcula a média ponderada de cada participante usando apenas os critérios indicados em pesos.

2. Atribui zero aos critérios em falta.

3. Ignora os critérios presentes nos participantes que não tenham um peso associado.

4. Ordena os participantes por pontuação decrescente e, nos empates, pelo nome.

5. Devolve a classificação como lista de tuplos (nome, pontuacao) e um dicionário que associe cada nome à lista de critérios em falta.

Os nomes são únicos. Os pesos devem ser não negativos e a sua soma deve ser positiva; caso contrário, lança ValueError. Arredonda apenas as pontuações finais a duas casas decimais.

```
classificacao, em_falta = classificar_participantes(
       ana, bruno, carla,
       teoria=0.4,
       pratica=0.6
 )
```

Resultado esperado:

```
classificacao == [
       ("Ana", 17.2),
       ("Bruno", 15.6),
       ("Carla", 6.0)
 ]

 em_falta == {
       "Ana": [],
       "Bruno": [],
       "Carla": ["pratica"]
 }
```